Linux for Beginners

A Practical Guide

Chapter 5 of 12

Installing & Managing Software with apt

Contents

Chapter 5

Installing & Managing Software with apt

5.1 What Is a Package Manager?

On Windows or macOS, installing software means visiting a website, downloading an installer, running it, clicking through prompts, and hoping it doesn't bundle unwanted extras. On Linux, there's a better way: the **package manager**.

A package manager is a system that handles the entire software lifecycle — finding, downloading, installing, updating, and removing software — from the command line. It resolves dependencies automatically (if program A needs library B, it installs B too), verifies cryptographic signatures to ensure nothing was tampered with, and keeps a consistent record of everything installed on your system.

On Ubuntu and all Debian-based distributions, the package manager is **apt** (**A**dvanced **P**ackage **T**ool). It's one of the most important tools you'll use as a Linux user.

Why package managers are better than manual installs

Concern	Manual install (Windows/macOS style)	Package manager (apt)
Finding software	Search web, find download page	apt search
Security	Hope the website is legitimate	Packages are cryptographically signed
Dependencies	Install each dependency yourself	Resolved and installed automatically
Updates	Check each app's website separately	sudo apt upgrade updates everything
Removal	Uninstaller may leave files behind	apt remove cleans up properly
Reproducibility	Hard to replicate on another machine	apt install list is scriptable

5.2 How apt Works: Repositories and the Package Index

apt doesn't download software from random websites. It downloads from curated **repositories** (repos) — servers maintained by Ubuntu, Canonical, or trusted third parties. Each package in a repository is tested, signed, and tracked.

Your system keeps a local **package index** — a database of all packages available across all your configured repositories, including their current version numbers and descriptions. This index is what apt searches when you look for software.

Repository configuration

Your repository list lives in `/etc/apt/sources.list` and the directory `/etc/apt/sources.list.d/`. You rarely need to edit these manually — `apt` commands handle it — but it's useful to know where they are:

```bash
$ cat /etc/apt/sources.list
deb http://archive.ubuntu.com/ubuntu jammy main restricted
deb http://archive.ubuntu.com/ubuntu jammy-updates main restricted
deb http://security.ubuntu.com/ubuntu jammy-security main restricted

# List all configured repos:
$ apt policy
```

The four Ubuntu repository components

Component	Contents	Support level
main	Free, open-source software officially supported by Canonical	Full support
restricted	Proprietary drivers supported by Canonical	Full support
universe	Free, open-source software maintained by the community	Community support
multiverse	Software restricted by copyright or legal issues	No official support

5.3 Updating the Package Index

Before installing or upgrading anything, always update your local package index. This downloads the latest package lists from all your repositories so apt knows what's currently available and what versions exist.

```bash
$ sudo apt update
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [119 kB]
Get:3 http://security.ubuntu.com/ubuntu jammy-security InRelease [110 kB]
Fetched 512 kB in 2s (256 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
42 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

> **TIP**
> sudo apt update does NOT install or upgrade anything. It only refreshes the list of available packages. Think of it as syncing a catalogue before shopping.

5.4 Installing Software

Once your package index is up to date, installing software is a single command. apt automatically resolves and installs all dependencies.

Basic installation

```bash
$ sudo apt install curl
Reading package lists... Done
Building dependency tree... Done
The following NEW packages will be installed:
  curl libcurl4
0 upgraded, 2 newly installed, 0 to remove and 42 not upgraded.
Need to get 194 kB of archives.
After this operation, 456 kB of additional disk space will be used.
Do you want to continue? [Y/n] Y
```

Installing multiple packages at once

```bash
$ sudo apt install git curl wget htop tree unzip
# Installs all six packages in one command
```

Useful install options

Option	Meaning	Example
-y	Automatically answer yes to all prompts	sudo apt install curl -y
--no-install-recommends	Skip optional recommended packages (smaller install)	sudo apt install nginx --no-install-recommends
--dry-run	Show what would be installed without doing it	sudo apt install vim --dry-run
=version	Install a specific version of a package	sudo apt install nginx=1.18.0-0ubuntu1

> **TIP**
> Always run sudo apt update before installing a package you haven't installed before. Without a fresh index, apt may try to download an outdated version or fail to find the package.

5.5 Removing Software

apt gives you two levels of removal: `remove` uninstalls the program but keeps its configuration files. `purge` removes everything including configuration — useful when you want a clean slate.

```bash
$ sudo apt remove curl            # remove package, keep config files
$ sudo apt purge curl             # remove package AND config files
$ sudo apt remove curl -y         # skip confirmation prompt

# Remove packages that were auto-installed as dependencies
# and are no longer needed by anything:
$ sudo apt autoremove
$ sudo apt autoremove --purge     # also remove their config files
```

Command	Removes binary?	Removes config?	When to use
apt remove	Yes	No	Want to reinstall later with same settings
apt purge	Yes	Yes	Fresh removal — don't want leftover config
apt autoremove	Yes	No	Clean up orphaned dependency packages

5.6 Upgrading Installed Packages

One of apt's greatest strengths: updating every installed package on your system at once. This is something that would take hours on Windows — on Linux, it's two commands.

```bash
# Step 1: refresh the package index
$ sudo apt update

# Step 2: upgrade all packages that have newer versions:
$ sudo apt upgrade
42 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
Need to get 87.4 MB of archives.
After this operation, 1,024 kB of additional disk space will be used.
Do you want to continue? [Y/n]
```

upgrade vs full-upgrade

Command	What it does	When to use
apt upgrade	Upgrades packages but never removes anything	Safe — for regular maintenance
apt full-upgrade	Upgrades packages and removes others if needed	When apt upgrade says packages are held back
apt dist-upgrade	Alias for full-upgrade (older name)	Same as full-upgrade

> **TIP**
> The two-command update routine — sudo apt update && sudo apt upgrade — is something every Linux user runs regularly. On servers, this is often automated with unattended-upgrades.

5.7 Searching for Packages

You don't always know the exact package name. apt gives you powerful search tools to find what you need without leaving the terminal.

```bash
# Search for packages matching a keyword:
$ apt search video editor
Sorting... Done
Full Text Search... Done
kdenlive/jammy 21.12.3-0ubuntu1 amd64
  non-linear video editor

openshot-qt/jammy 2.6.1-1 amd64
  create and edit videos and movies

# Search only in package names (faster, more precise):
$ apt search --names-only python

# Show full details about a specific package:
$ apt show curl
Package: curl
Version: 7.81.0-1ubuntu1.10
Depends: libc6, libcurl4
Description: command line tool for transferring data with URL syntax
...
```

> **TIP**
> apt search searches package names AND descriptions. If you get too many results, add --names-only to search just the package name, or pipe through grep: apt search python | grep '^python3'

5.8 Inspecting Packages

Before installing a package — or when troubleshooting — you often want to know more about what's installed and what files it placed on your system.

```bash
# Show detailed info about a package (installed or available):
$ apt show nginx

# List all installed packages:
$ apt list --installed

# List packages that can be upgraded:
$ apt list --upgradable

# Show which package owns a specific file:
$ dpkg -S /usr/bin/curl
curl: /usr/bin/curl

# List all files installed by a package:
$ dpkg -L curl
/.
/usr
/usr/bin
/usr/bin/curl
/usr/share/doc/curl
...

# Check if a specific package is installed:
$ dpkg -l curl
||/ Name    Version                 Architecture Description
+++-=======-=======================-============-==================
ii  curl    7.81.0-1ubuntu1.10      amd64        command line tool...
```

5.9 apt vs apt-get: Which to Use

You'll encounter both `apt` and `apt-get` in documentation, tutorials, and Stack Overflow answers. Understanding the difference prevents confusion.

Feature	apt	apt-get
Introduced	Ubuntu 16.04 / Debian 8 (2014)	Much older (1998)
Progress bar	Yes — shows download progress	No
Coloured output	Yes	No
Human-friendly	Designed for interactive use	Designed for scripts
Commands	apt install / remove / search	apt-get install / remove / apt-cache search
Script use	Not recommended (output may change)	Stable — safe in shell scripts
Recommendation	Use in the terminal day-to-day	Use in scripts and automation

> **TIP**
> Rule of thumb: use apt when you're typing in a terminal. Use apt-get in shell scripts and automated tasks — its output format is guaranteed stable across versions.

5.10 Adding Third-Party Repositories and PPAs

Ubuntu's official repositories are curated and stable, but they don't have everything — and they may have older versions of software that moves quickly (like Node.js or Docker). Third-party repositories and **PPAs** (Personal Package Archives) extend what apt can install.

Adding a PPA

PPAs are hosted on Launchpad and maintained by developers or teams outside of Canonical. They're a common way to get newer versions of software on Ubuntu:

```bash
# Example: add the PPA for a newer version of git:
$ sudo add-apt-repository ppa:git-core/ppa
$ sudo apt update
$ sudo apt install git

# Remove a PPA:
$ sudo add-apt-repository --remove ppa:git-core/ppa
```

Adding a third-party repo (e.g. Docker)

Major software projects like Docker, Node.js, and VS Code provide their own apt repositories. The process always follows the same four steps:

1. Install prerequisites
2. Add the repository's GPG signing key (so apt can verify packages)
3. Add the repository URL to your sources list
4. Run apt update and install

```bash
# Example: adding Docker's official repository
$ sudo apt install ca-certificates curl gnupg
$ sudo install -m 0755 -d /etc/apt/keyrings
$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
    | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
$ echo "deb [arch=$(dpkg --print-architecture) \
    signed-by=/etc/apt/keyrings/docker.gpg] \
    https://download.docker.com/linux/ubuntu \
    $(. /etc/os-release && echo $VERSION_CODENAME) stable" \
    | sudo tee /etc/apt/sources.list.d/docker.list
$ sudo apt update
$ sudo apt install docker-ce docker-ce-cli
```

> **WARNING**
> Only add PPAs and third-party repos from sources you trust. A malicious repository could push packages with root-level access to your system. Stick to official project repos and well-known PPAs.

5.11 Installing from Source When No Package Exists

Occasionally, the software you need isn't in any repository. In that case, you can compile and install it from source code. This is more involved, but the process is standard across most open-source projects.

The classic configure / make / make install pattern

```bash
# 1. Install essential build tools:
$ sudo apt install build-essential

# 2. Download the source code (usually a .tar.gz archive):
$ wget https://example.com/software-1.0.tar.gz
$ tar -xzf software-1.0.tar.gz
$ cd software-1.0/

# 3. Configure the build for your system:
$ ./configure

# 4. Compile the source code:
$ make

# 5. Install the compiled binary:
$ sudo make install
```

Modern alternatives to source compilation

Source compilation is becoming less common thanks to newer universal package formats that work across all Linux distributions:

Format	Command	Advantage
Snap	sudo snap install	Sandboxed; auto-updated; works across distros
Flatpak	flatpak install	Sandboxed; wide app library via Flathub
AppImage	chmod +x app.AppImage && ./app.AppImage	No install needed; portable single file

5.12 Practical Software Management Workflow

Here's a realistic sequence covering a full software management session — from setting up a fresh system to keeping it maintained.

```bash
# ■■ Setting up a fresh Ubuntu system ■■■■■■■■■■■■■■■■■■■■■■■■■■■

# 1. First: refresh the package index
$ sudo apt update

# 2. Upgrade all existing packages
$ sudo apt upgrade -y

# 3. Install your essential tools
$ sudo apt install -y git curl wget vim htop tree unzip

# 4. Install a development stack example (Python)
$ sudo apt install -y python3 python3-pip python3-venv

# 5. Verify installs
$ git --version && python3 --version && curl --version

# ■■ Regular maintenance (run weekly or monthly) ■■■■■■■■■■■■■■■■■■

# Refresh index + upgrade all packages in one line:
$ sudo apt update && sudo apt upgrade -y

# Clean up packages no longer needed:
$ sudo apt autoremove -y

# Clean the local package download cache:
$ sudo apt clean

# See how much disk space apt cache is using:
$ du -sh /var/cache/apt/archives/
```

5.13 Essential Commands: Quick Reference

Command	What It Does	Notes
sudo apt update	Refresh the package index	Always run before install/upgrade
sudo apt upgrade	Upgrade all installed packages	Safe — never removes packages
sudo apt full-upgrade	Upgrade + remove packages if needed	Use when packages are held back
sudo apt install	Install a package	Add -y to skip confirmation

Command	What It Does	Notes
sudo apt install -y	Install without confirmation prompt	Good for scripts
sudo apt remove	Remove a package (keep config)	Config files stay behind
sudo apt purge	Remove package and config files	Clean uninstall
sudo apt autoremove	Remove unused dependency packages	Run after remove/purge
sudo apt clean	Clear downloaded package files from cache	Frees disk space
apt search	Search for packages by keyword	No sudo needed
apt show	Show details about a package	Version, deps, description
apt list --installed	List all installed packages	Pipe to grep to filter
apt list --upgradable	List packages with available updates	See what upgrade will touch
dpkg -l	Check if a package is installed	ii = installed
dpkg -L	List files installed by a package	See where files went
dpkg -S	Find which package owns a file	dpkg -S /usr/bin/curl
sudo add-apt-repository ppa:	Add a PPA	Then sudo apt update
sudo snap install	Install a Snap package	Alternative to apt

5.14 Chapter Summary

✓ A **package manager** handles the entire software lifecycle — find, install, update, remove — with dependency resolution, signature verification, and full records.

✓ **apt** is Ubuntu's package manager. It downloads from curated, cryptographically signed **repositories** — not random websites.

✓ `sudo apt update` refreshes the index. It does NOT install or upgrade anything — it only syncs the catalogue.

✓ `sudo apt install` installs software and all its dependencies automatically. Use `-y` to skip prompts.

✓ `apt remove` uninstalls but keeps config. `apt purge` removes everything. `apt autoremove` cleans up orphaned dependencies.

✓ The two-command maintenance routine: `sudo apt update && sudo apt upgrade`. Run it regularly.

✓ `apt search`, `apt show`, and `dpkg -l / -L / -S` let you find and inspect packages without installing anything.

✓ Use **apt** interactively in the terminal. Use **apt-get** in scripts — its output is stable and guaranteed not to change.

✓ **PPAs** and third-party repos extend what apt can install. Only add repos from sources you trust — they have root access to your system.

✓ When no package exists: compile from source (`configure / make / make install`) or use **Snap**, **Flatpak**, or **AppImage**.

What's Coming in Chapter 6

You can now install any software on Linux. Chapter 6 focuses on a skill you'll use every single day: editing files directly in the terminal. You'll learn:

- Why terminal text editors matter — even if you prefer graphical editors
- **nano** — the beginner-friendly editor: open, edit, save, exit
- **vim** — the powerful modal editor: modes, navigation, and basic editing
- How to choose between nano and vim for different situations
- Editing system configuration files safely
- Viewing files without editing: `cat`, `less`, `head`, `tail` revisited

Chapter 6 is where you stop depending on graphical apps to edit files.

Author

Ashik A

github.com/ashihh07

This guide was created, organized, refined, and designed by Ashik A using GitHub documentation, industry best practices, publicly available learning resources, and AI-assisted research.